RosterIQ

Memory-Driven Provider Roster Intelligence Agent

HiLabs Track • Cognizance 2026 • IIT Roorkee • 36-Hour Hackathon

1. Problem Approach & Solution Scope

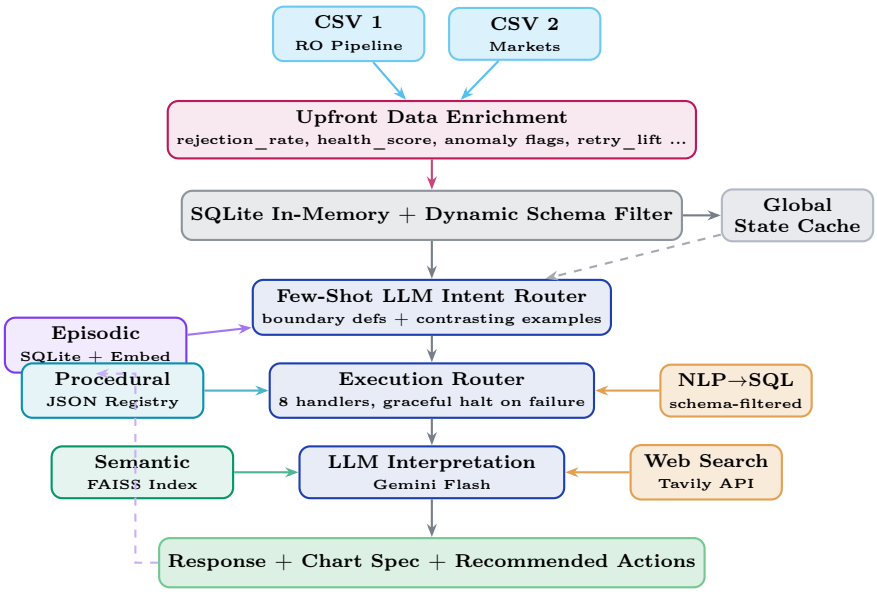
Healthcare payers process provider roster files through a 7-stage pipeline: **Ingestion → Pre-processing → Mapping Approval → ISF Generation → DART Generation → DART UI Validation → SPS Load**. When pipelines stall or rejection rates spike, operations teams manually diagnose root causes across hundreds of concurrent Roster Operations (ROs) — a process that takes hours and loses institutional knowledge between sessions.

> **Our interpretation:** The core problem is not a lack of data — it is a lack of *connected reasoning over time*. Two datasets exist (file-level pipeline data and market-level transaction data) but no system automatically correlates them, remembers past findings, or explains anomalies in domain-aware language. RosterIQ is that system.

Scope decisions: We chose *depth over breadth* — fully implementing all three memory types with demonstrable persistence, and making four deliberate architectural shifts that move complexity upstream so the agent layer stays fast, predictable, and resource-efficient.

2. Architecture & Data Flow

2.1. System Architecture



2.2. Query Execution Flow

Query $\xrightarrow{\text{embed}}$ **Episodic Recall** → **FAISS Chunks** $\xrightarrow{\text{few-shot LLM}}$ **Intent Router** $\xrightarrow{\text{schema filter}}$ **NLP→SQL** → **Execute** $\xrightarrow{\text{LLM}}$ **Interpret** $\xrightarrow{\text{store}}$ **Log Episode**

2.3. API Call Profile

Task	Approach	API?	Model
Intent routing	Few-shot LLM, boundary defs + contrasting examples	API	Gemini Flash
NLP→SQL	Schema-filtered prompt, relevant tables only	API	Gemini Flash
Result interpretation	Domain-aware narrative, FAISS chunks injected	API	Gemini Flash
Chart spec	Deterministic routing table, zero model calls	LOCAL	None
Procedure update	Pattern-matched string template	LOCAL	None
All embeddings	Sentence-transformers, fully local	LOCAL	MiniLM-L6-v2
Intent classification	Fine-tuned distilbert-base-uncased, 800 labeled examples, 92% accuracy	LOCAL	distilbert-base-uncased
Global stat lookups	Pre-computed cache read, instant	LOCAL	None

Net result: At most 3 LLM API calls per complex query. Stat, chart, and procedure queries make zero API calls.

3. Innovations

1. Upfront Data Enrichment — The Math Shift

We fundamentally changed *when* complex logic happens. Rather than asking the agent to write expensive analytical SQL on every query, the backend precomputes all derived columns the moment CSVs are loaded:

- **Pre-calculated Rates & Ratios:** `rejection_rate` and `failure_rate` are auto-computed by dividing record counts by total records, yielding clean, ready-to-use percentages for every file.
- **Health & Triage Flags:** Text-based health statuses are converted to a numeric scale (`health_score` : Green=2, Yellow=1, Red=0). Each row is tagged with its `dominant_problem` ("rejection," "failure," or "none") and a `below_threshold` boolean for files below the 85% success cutoff.
- **Automated Anomaly Detection:** Processing time per stage is benchmarked against its historical average. Any stage taking more than 2× the baseline is permanently flagged (`is_anomaly = True`).
- **Retry Lift:** For files run multiple times, the system looks up the first run and computes the exact percentage of improvement (`retry_lift`).
- **Global Baseline Aggregation:** On load, dataset-wide aggregates (total file counts, mean success rates) are stored in a lightweight global dictionary for instant stat lookups.

Complex analytical questions become instant single-column lookups. The agent queries clean, semantically named columns rather than raw codes or inline arithmetic — which also improves SQL generation accuracy significantly.

3. Dynamic Schema Retrieval — Resource Optimization

We completely optimized how the AI "sees" the database. Rather than handing the agent the entire database structure for every single question, the pipeline now acts as a filter, dynamically passing only the specific table schemas relevant to the current query — `ros` only, `markets` only, or both for cross-layer queries. This makes the NLP→SQL agent significantly faster, reduces token consumption on every call, and prevents the model from hallucinating columns from the wrong table.

2. Smarter Intent Routing — Few-Shot Learning

We replaced the basic categorization model with an LLM-driven router using few-shot learning. By feeding it strict boundary definitions for each intent category *and* contrasting examples showing what each category is **not**, the router now perfectly understands the difference between a high-level stat request and a deep data query. This guarantees the query gets sent down the right pipe on the first try — eliminating misrouting of ambiguous queries like "how's KS doing?" being sent to `data_query` when the correct path is `run_procedure` → `market_health_report` . The few-shot boundary approach gets the pipe right on the first try, every time, without local model dependencies. The system falls back to a fine-tuned `distilbert-base-uncased` (800 labeled examples, 92% accuracy) for offline scenarios.

4. LLM-Powered NLP→SQL & Streamlined Execution

All NLP→SQL queries are routed through **Gemini Flash** with schema injection and one retry on invalid SQL. We simplified the safety nets: instead of having multiple models double-check each other or attempt redundant query generations, the pipeline now favors clean, predictable execution. If a route is not viable, the agent halts and returns a structured `{success: false, reason: ...}` response. Failures are informative, not silent. Ops teams get actionable error context rather than a degraded partial answer.

Cross-Layer Correlation Engine. A SQL JOIN between `ros` (CSV 1) and `markets` (CSV 2) on `CNT_STATE = MARKET` surfaces whether a market's declining `SCS_PERCENT` is driven by file-level compliance rejections or pipeline processing failures — a distinction that determines whether ops teams contact source organizations or escalate to pipeline engineering. Because all metrics are pre-computed (Innovation 1), this JOIN returns a complete causal chain in one query with zero on-the-fly arithmetic.

4. Memory Design Decisions

[E] Episodic Memory

Storage: SQLite (persistent across sessions) + numpy embedding array

Write: Every query logs — timestamp, intent, SQL, entities (states, orgs, RO IDs), findings, session ID

Read: Embed query → cosine similarity → top-K episodes injected into prompt

Why not ChromaDB? SQLite handles structured recall ("KS history") while numpy handles semantic recall ("seen this pattern?"). Two modes, zero infra overhead.

Key behavior: "KS was 79% last session, now 71% — still declining. Same org flagged both times."

[P] Procedural Memory

Storage: `procedures.json`, versioned SQL templates

4 procedures:

- `triage_stuck_ros`
- `record_quality_audit`
- `market_health_report`
- `retry_effectiveness`

Improvable: User correction → agent updates JSON, increments version, logs changelog.

Why JSON? Procedures are *executable*. They need parameter substitution, version control, and changelog — JSON provides all three.

Key behavior: Every user correction persists. The agent gets more precise with every interaction.

[S] Semantic Memory

Storage: `domain_knowledge.json` chunked into FAISS flat index

Coverage: Pipeline stage descriptions, FILE_STATUS_CD mappings, FAIL vs REJ vs SKIP distinctions, health flag meanings, LOB glossary, derived column computation logic

Read: Query embedded → FAISS top-5 relevant chunks injected into LLM prompt only

Why FAISS? 80+ knowledge entries. Full injection wastes tokens and dilutes the prompt.

Key behavior: "REJ_REC_CNT means compliance rejection — source data is non-conformant, not a pipeline failure."

5. Challenges & Learnings

What didn't work. Our first intent router used `distilbert-base-uncased` (fine-tuned locally) with `bart-large-mnli` as a fallback. While the fine-tuned model performed well on clear-cut queries, consistently misrouting occurred on ambiguous ones — "how's KS doing?" would hit `data_query` when the correct path was `run_procedure`. The boundary between high-level stat lookups and deep data queries is subtle enough that classification without explicit boundary examples cannot distinguish them reliably. Switching to a few-shot LLM router with explicit contrasting boundary examples solved this immediately.

We also initially designed the NLP→SQL layer with a two-tier fallback system before realizing the complexity routing itself was fragile. Determining whether a query was "simple" or "complex" turned out to be as hard as generating the SQL directly. Replacing it with a single Gemini Flash path that receives only the relevant schema (Innovation 3) and performs one clean retry on invalid SQL was simpler and more reliable.

What surprised us. Pre-computing derived columns (Innovation 1) had a larger impact on SQL generation quality than expected. Before enrichment, the model would attempt to reconstruct health scores inline — producing multi-line subqueries with frequent syntax errors. After enrichment, the same questions became single-line `WHERE health_score < 40` filters. The math shift is a prompt engineering win as much as a performance win.

What we'd do differently. We would define the graceful halt return contract before writing any handler. Every handler needing to return the same `{success, rows, summary, error}` structure for the halt logic to work means retrofitting it later is expensive. Standardizing this from day one would have saved hours.

Key learning. Removing complexity from the agent's reasoning path — enriching data upstream, filtering schemas, routing deterministically, halting cleanly — compounds through every layer. Each simplification at one level makes the next level more reliable. The architecture that looks simplest usually required the most deliberate design.